

One Table, Six Trades, and Nothing Invented

A formation academy, a tailor, a courier and a photographer now open a shop through the same schema, the same authorization and the same discovery queries — differing only in the words on screen. And the marketplace that lists them shows **no rating, no follower count, no verification badge and no sales figure**, because Afrinext has none of those facts and a buyer in Niamey is the wrong person to show an invented number to.

Branch **claude/create-app-repository-1cxsih**

HEAD **440fe63**

Unit tests **583 passed**

Browser tests **39 passed**

Mutations killed **26 / 26**

CI **3 / 3 green**

Ledger touched **none**

WHAT THIS MILESTONE IS, IN ONE PARAGRAPH

The product side of Afrinext was a fake-data prototype: seeded listings, invented star ratings, a "saved" list in `localStorage`. All of it is deleted. In its place is a **store engine** — one `stores` table with a validated `store_type`, a lifecycle with a moderation path, visibility decided in SQL, and a marketplace that reads only facts the database actually holds.

No money moved and no ledger row was written. Settlement, payouts, refund execution and the payment state machine are untouched — see section H.

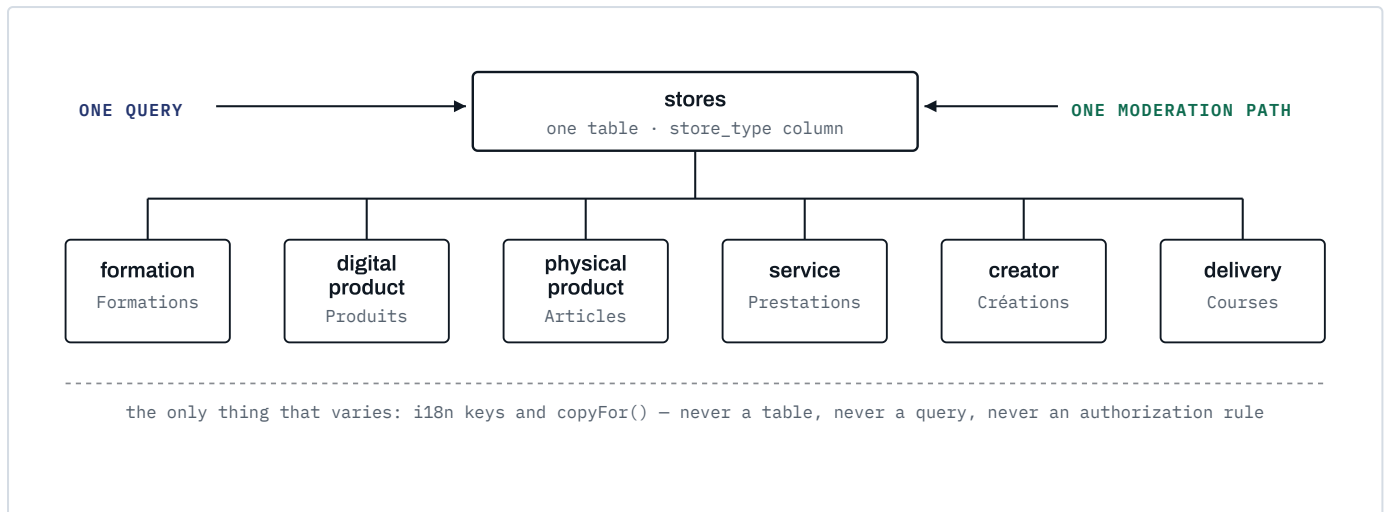
A

Six businesses, one entity, and why that is not a shortcut

The brief named six things Afrinext must sell: formations, digital products, physical products, services, creator work and delivery. The tempting reading is six features. It is one.

A training academy in Niamey and a mechanic on the Route de Tillabéri need the same things from a marketplace: an identity buyers can find, a list of what they offer, a price in XOF, an owner who controls it, and a moderator who can stop it. What differs is **vocabulary and presentation** — "formations" versus "prestations" — not structure.

Six near-identical tables would have to be kept in step forever. Every feature after this one — moderation, search, commission, payouts, suspension — would be written six times, and the sixth copy would be the one with the bug. So the type is a **column**, and the differences live in a presentation layer that cannot reach a query.



The engine. Adding a seventh trade is a new value in `STORE_TYPES`, the `CHECK` constraint and the i18n catalogues. The exhaustive `Record` in `store-presentation.ts` fails the build until somebody decides what its offerings are called – a dynamic key would have compiled and shipped a page labelled `offering.auction`.

There is no default type, and that is a product decision

`parseStoreType` throws for anything not on the list – including `undefined`, `""`, `"FORMATION"` and `"service "`. It never falls back.

The type decides how somebody's shop presents itself to buyers. Guessing it means guessing their business. A seller who does not answer the question does not get a store, and the wizard's first step cannot be skipped: the **Continue** button is disabled until one of the six is chosen.

TRANSITION	WHO	PERMISSION	NOTE
→ <code>draft</code>	the creator	<code>store.create</code>	owner is the session, never the input
<code>draft</code> → <code>published</code>	owner	<code>store.update</code>	stamps <code>published_at</code> once
<code>published</code> → <code>draft</code>	owner	<code>store.update</code>	<code>unpublishStore</code>
any → <code>suspended</code>	moderator	<code>store.moderate</code>	reason of ≥ 4 characters, audited
<code>suspended</code> → <code>draft</code>	moderator	<code>store.moderate</code>	not to published
<code>suspended</code> → <code>published</code>	nobody	—	the SQL excludes it

Three of these carry a reason worth stating.

Republishing cannot jump the queue

```
published_at = coalesce(published_at, now()),
-- not now(): the first publish stamps it and later ones leave it alone
```

"Newest stores" is the most valuable position on the marketplace home. With a bare `now()`, a seller could hold it indefinitely by unpublishing and republishing every morning. The `coalesce` is what makes that date mean *when this store first became public*.

Reinstatement returns a store to draft, not to published

Lifting a suspension restores the owner's *ability* to be public; it does not publish on their behalf. A seller whose store was suspended over a disputed listing should decide when it goes back up — not discover that a moderator did it for them, with the old listing still on it.

The guards are in the statement, not before it

```
update stores set status = 'published', ...
  where id = $1 and status <> 'suspended'

store_type = case when status = 'draft' then coalesce($1, store_type)
              else store_type end
```

A read followed by a check leaves a gap: a suspension landing between the two would be overwritten by a publish that had already decided the store was fine. Putting both conditions inside the `UPDATE` closes it — the row either matches or the statement affects nothing, and the caller is told so.

The second is why a published store's **kind cannot change**. Buyers have seen it presented one way and its offerings were written for that presentation. Turning a formation into a delivery service is not an edit; it is a different business.

OWNERSHIP COMES FROM THE SESSION AND FROM NOWHERE ELSE

`owner_user_id` is `actor.userId`. There is no field in any request through which somebody could open a store in another person's name — and a test posts `ownerUserId`, `owner_user_id` and `userId` together, all naming a second seller, to prove the row still belongs to the sender.

`store.moderate` is deliberately a **platform** permission: owning a store is not it, so a seller cannot suspend a rival, or themselves.

Visibility is decided in SQL, or it is not decided

Every public query filters `status = 'published'` in the statement. A draft store is not a row the caller receives and declines to render. It is a row the caller never receives.

FUNCTION	FILTER
<code>findPublicStore</code>	<code>slug = \$1 and status = 'published'</code>
<code>listPublicProducts</code>	<code>p.status = 'published' and s.status = 'published'</code>
<code>discoverStores</code>	<code>s.status = 'published'</code>
<code>discoverOfferings</code>	<code>p.status = 'published' and s.status = 'published'</code>
<code>countStoresByType</code>	<code>status = 'published'</code>

Both conditions on the offering queries, always. A published product inside a suspended store is not public, and the join enforces that rather than trusting a caller to check the store separately. The browser test proves it from outside: it publishes a store, publishes a product in it, confirms both are reachable anonymously, suspends the store, and then asserts the **product** is gone from discovery too.

"Not published" and "does not exist" are the same answer

`findPublicStore` returns `undefined` for a draft store, a suspended store and a slug nobody ever took. The route turns all three into a 404. Telling them apart would let a stranger enumerate which slugs are in use and which sellers have been suspended.

THE PUBLIC PROJECTION IS NOT A CAST

`PublicStore` is built field by field from `StoreRecord`, which carries the owner's user id and the internal store id. Naming the ten public fields is what stops a column added next year — an internal note, a moderation flag, a risk score — from reaching a stranger's browser because somebody ran a migration. Mutation S5 replaces the projection with a cast; a test fails.

A defect the type system introduced, and the fix

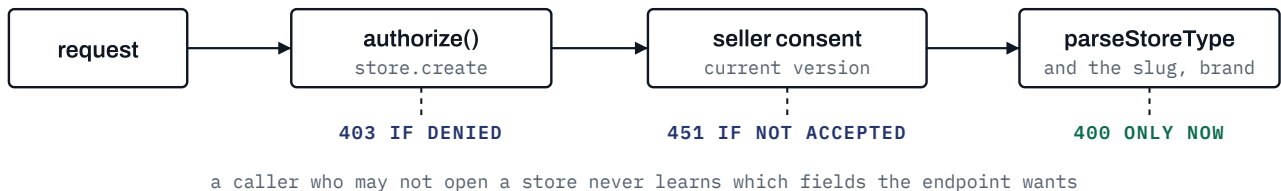
This one is worth reading in full, because the cause was the opposite of careless.

`CreateStoreInput.storeType` was typed `StoreType` — the strictest thing available. That forced every HTTP boundary to call `parseStoreType` to satisfy the compiler, which put **input validation before authorization**:

```
// before – the compiler required this, and this is the bug
await catalog.createStore(db, actor, {
  storeType: catalog.parseStoreType(body.storeType), // throws first
  ...
}); // authorize() never reached
```

A stranger posting to `POST /api/v1/stores` was answered **400 "unsupported store type"** instead of **403 "denied"** — a fact about the API's shape that they had not earned. The browser suite caught it: three consent tests and one catalog test went red the moment `storeType` became required, all reporting 400 where they expected 403 or 451.

The fix is to type the field **raw** and let the domain parse it, after both gates:



The order that must hold. Four assertions now hold it: two domain tests post a store type of `"not_a_type"` from an actor with no permission and from one who has not accepted the terms, and require `PermissionDeniedError` and `consent.required` respectively; the catalog browser test posts a body with **no name at all** and requires 403; the consent test posts one with **no store type** and requires 451.

REVIEWER'S NOTE — THE TYPE ANNOTATION IS DELIBERATELY WEAKER

`storeType: StoreType | (string & {})` looks like a loosening and reads like a mistake. It is neither. The comment at the field says why, and mutation **S11** — which moves the parse back above the gates — is killed by two tests. The value is still validated; it is validated in the one place that can be sure the gates have already run.

Nothing on this marketplace is invented

The brief was explicit, and so is the code. There is no rating, no review count, no follower total, no "verified" badge, no seeded `featured` flag, no view counter and no random ordering anywhere in `marketplace.ts` or in the interface that reads it.

Two facts are real, and those are the two shown: **when a store was published**, and **how many published offerings it has** — including zero, said plainly as "Aucune offre pour l'instant".

SORT	ORDERING	WHAT IT READS
<code>newest</code> (default)	<code>published_at desc nulls last</code>	a real timestamp
<code>popular</code>	<code>count(paid orders) desc, published_at desc</code>	<code>orders.status = 'paid'</code> , nothing else

A pending checkout, an expired one and a failed one are not sales. Counting them would measure interest and call it demand — so the test inserts three of exactly those against one store, asserts the ranking does not move, then adds a single `paid` order to the other store and asserts it now outranks all three.

THE ROW THAT IS NOT SHOWN

The marketplace home does not render the "popular" row **at all** until it would differ from the newest row. With no sales, "popular" is recency wearing a different label, and presenting it that way is precisely the fabrication this section exists to avoid. The screenshots in section F were taken against a database with no paid orders, which is why that row is absent from them.

The counts on the category tiles

Mutation S6 made `countStoresByType` include drafts and suspensions, and **no test failed**. That was a real gap: the home page's category tiles would have advertised a marketplace larger than the one that exists, every extra being a shop nobody can open — the same fabrication as a fake rating, arriving through arithmetic. A test now creates one live creator store, one draft and one suspended, and requires the count to be 1.

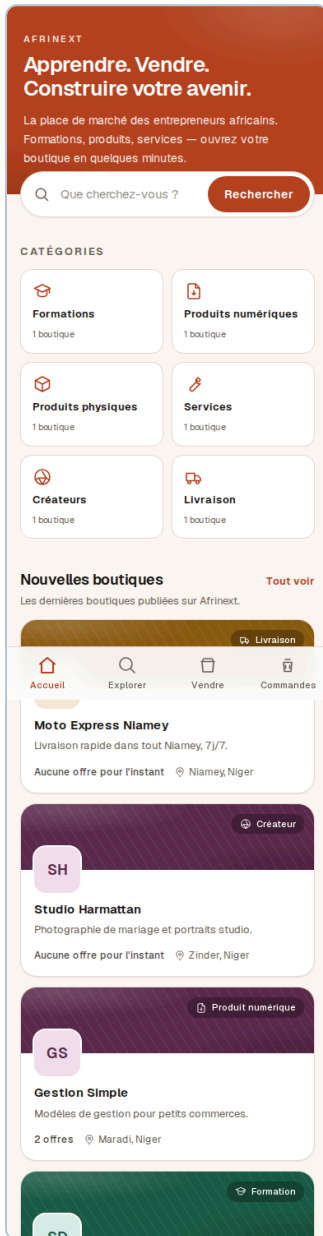
Search, and its bound

`ILIKE` over name, tagline and description: the honest tool for a marketplace with tens of stores — nothing extra to run, nothing to keep warm, predictable in French. The pattern is escaped, so a buyer searching for `100%` gets that string rather than a wildcard; the test names one store "*Promo 100% coton*" and requires that search to return exactly it and not the other. `limit` is clamped to 1–48 and `offset` to 0–5000, so no URL asks for the whole marketplace at once.

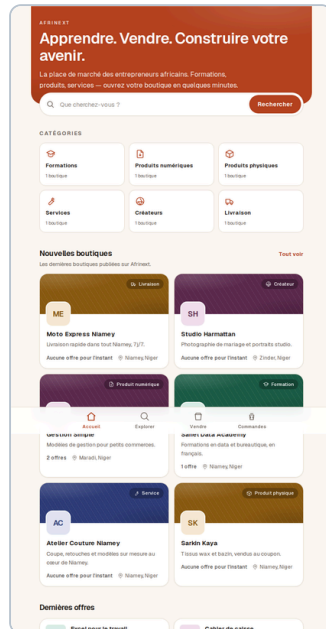
The interface, as it actually renders

Every screenshot below is a real page from a production build, served by `next start` against PostgreSQL. All four widths — 390, 768, 1280 and 1680 — were inspected, and a script asserts **zero horizontal overflow** at every one of them on every page shown.

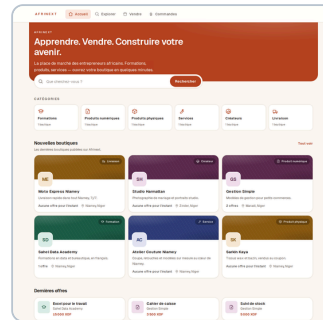
The marketplace



390 px. Hero, search, six real category counts, then stores. One column.



768 px. Categories to three across, store cards to two.



1280 px. Six categories in a row, three cards across — and the tab bar has moved to the top.

Each card carries a brand-washed cover, the store's monogram, its type and its *real* offering count. The six palettes — laterite, indigo, forest, ochre, aubergine, sable — are named for materials rather than hex values, and a store that chooses none gets one deterministically from its slug, so a shop is never unstyled and always looks the same.

AFRICAN IDENTITY THROUGH RESTRAINT

The brief warned against tribal decoration, random flags and stereotype. What is here instead: a palette taken from laterite earth, indigo dye, harmattan ochre and sahel green; a diagonal weave in the covers at low opacity, reading as woven cloth rather than as a pattern library; generous type; French first, because the launch market is Niger. No mask, no map of Africa, no kente border.

Opening a store: four questions, not nine fields

The image displays four sequential mobile app screens for opening a store, titled 'Ouvrir une boutique'.

- Step 1:** 'Que souhaitez-vous proposer ?' (What do you want to propose?). It lists six categories: Formations (highlighted with a red border and a checkmark), Produits numériques, Produits physiques, Services, Créateurs, and Livraison. Each category has a brief description.
- Step 2:** 'Ouvrir une boutique'. It asks for the store name (e.g., 'Institut Sahel Numérique'), a presentation phrase (e.g., 'Bureautique, data et web, en français.'), a short description ('Une ligne pour dire ce que vous faites.'), and a color palette (e.g., a green circle with a checkmark).
- Step 3:** 'Voici votre boutique' (Here is your boutique). It shows a preview of the store's public page, including the name, description, and category.
- Step 4:** The final screen showing the completed store preview and a 'Créer ma boutique' button.

Step 1 — the trade.

Selection is border, tint and a check mark, never colour alone. **Continue** is disabled until one is chosen.

Step 2 — identity. Name, tagline, description, palette. No slug field: it is derived server-side and never sent.

Step 4 — the real thing.

The same cover, monogram and badge the public page renders, so publishing is never a leap of faith.

A single form with nine fields tells a first-time seller in Niamey that this will be hard. Four short screens with one question each tell them it will not. Nothing is submitted until the last step, so abandoning the flow leaves no half-built store behind — and the client state is a nicer way to fill a form, never a security boundary: the server validates every field again.

The seller's dashboard tells them what to do next

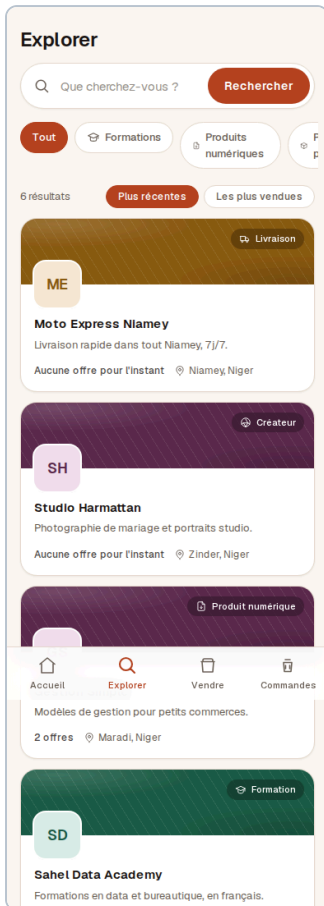
The screenshot shows the AFRINEXT seller dashboard for a store named 'Institut Sahel Numérique'. The top navigation bar includes links for 'Accueil', 'Explorer', 'Vendre' (highlighted), and 'Commandes'. The store's public address is listed as 'Pas encore publiée'. A 'PROCHAINE ÉTAPE' (Next Step) section prompts the user to 'Ajoutez votre première offre' (Add your first offer). Below this, the 'Formations' (Courses) section shows 'Aucune offre pour l'instant' (No offers for now) and encourages creating the first offer. A form titled 'Ajouter une formation' (Add a course) contains input fields for 'Titre du produit' (Product title), 'Résumé' (Summary), and 'Prix (XOF)' (Price in XOF), followed by an 'Ajouter une formation' button. At the bottom, a 'BIENTÔT DISPONIBLE' (Coming soon) section features four tiles for 'Commandes', 'Clients', 'Statistiques', and 'Paramètres', all marked as 'Bientôt disponible' (Coming soon).

A draft formation store. The offerings heading reads *Formations* and the form below it reads *Ajouter une formation* — the vocabulary follows the trade all the way down. "Prochaine étape" names one action. The four tiles at the bottom say **Bientôt disponible** rather than pretending to work.

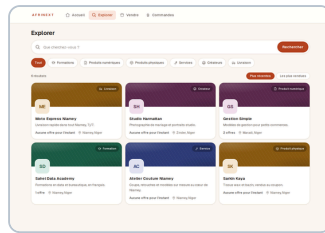
PUBLISHING COMES FIRST — SEE SECTION I

This shot was taken on a store that already has an offering, so its next step is the last one. On an *empty* draft the first step is "Publiez votre boutique", and the control is offered: review decision 1 made a store a commercial identity that can be published before it has stock. What that costs, and the seven assertions that pay for it, are in **section I**.

Search and browse, with no client JavaScript at all



390 px. Type filters scroll horizontally inside their own container; the page itself never does.



1280 px. The filters wrap instead of scrolling. Result count is announced with `aria-live`.



A public store page. No rating, no follower count, no badge. Contact is the number the owner entered for buyers — never their sign-in number.

Query, type, sort and page all live in URL search params. A result is therefore a link somebody can send to a friend, the back button steps through searches, and the whole screen renders on the server. That is not a purity exercise: this is a marketplace for phones on mobile data, and the fastest search result is one that arrives as HTML.

Twenty-six deliberate defects, each breaking one claim this milestone makes, applied to an isolated git worktree and run against the catalog suite. A mutation nothing catches is a claim nothing is holding.

S24–S26 are review decision 1's, and they are the reason that decision is safe to make. A permissive rule has exactly two ways to be implemented wrongly — re-imposing the requirement somewhere else, and filling the emptiness with something invented — so both are mutations, and both die.

#	THE DEFECT INTRODUCED	RESULT
S1	<code>findPublicStore</code> drops <code>status = 'published'</code> → a draft or suspended store becomes publicly reachable	KILLED · 6
S2	<code>listPublicProducts</code> stops requiring the STORE to be published → a suspended store's products stay public	KILLED · 1
S3	<code>discoverStores</code> drops the published filter → drafts appear in the marketplace listing	KILLED · 3
S4	<code>discoverOfferings</code> drops the store's published filter → offerings of a suspended store are discoverable	KILLED · 71
S5	<code>PublicStore</code> becomes a cast instead of a projection → owner id and internal store id reach the browser	KILLED · 4
S6	<code>countStoresByType</code> counts drafts and suspensions → the category tiles advertise unreachable stores	KILLED · 1
S7	<code>owner_user_id</code> taken from the request body → one seller opens a store in another's name	KILLED · 1
S8	<code>createStore</code> drops <code>authorize()</code> → anybody with an account opens a store	KILLED · 3
S9	<code>updateStore</code> drops <code>authorize()</code> → a stranger edits somebody else's store	KILLED · 2
S10	suspension needs only <code>store.update</code> on that store → an owner suspends their own store outside moderation	KILLED · 7
S11	the store type is parsed BEFORE the gates → an unauthorized caller is told the type is invalid, not that they are denied	KILLED · 2
S12	<code>parseStoreType</code> defaults instead of throwing → an unrecognised type silently becomes a digital product	KILLED · 2
S13	<code>isStoreType</code> matches case-insensitively → "FORMATION" is accepted and stored differently	KILLED · 2
S14	<code>parseStoreBrand</code> defaults instead of throwing → an unlisted palette silently becomes laterite	KILLED · 1

#	THE DEFECT INTRODUCED	RESULT
S15	a store's type stays editable after publication → a published formation becomes a delivery service	KILLED · 1
S16	<code>publishStore</code> drops <code>status <> 'suspended'</code> → a suspended seller republishes themselves	KILLED · 2
S17	<code>published_at = now()</code> instead of <code>coalesce</code> → republishing claims the top of "newest stores"	KILLED · 1
S18	<code>reinstateStore</code> goes straight to published → a moderator publishes on the owner's behalf	KILLED · 1
S19	suspension accepts an empty reason → the audit trail records no why	KILLED · 1
S20	"popular" counts every order, not just paid ones → interest is presented as demand	KILLED · 1
S21	the offering count includes drafts → a card claims offerings a buyer cannot buy	KILLED · 1
S22	the search pattern is not escaped → a search for <code>100%</code> matches everything	KILLED · 1
S23	the <code>limit</code> clamp is removed → one URL asks for the whole marketplace	KILLED · 1
S24	an empty store is refused publication → review decision 1 silently reverted	KILLED · 226 DECISION 1
S25	an empty store gets a placeholder offering → a product nobody created, priced at zero	KILLED · 3 DECISION 1
S26	zero offerings is reported as one → a card claims an offering that does not exist	KILLED · 2 DECISION 1

THE ONE SURVIVOR FROM THE FIRST RUN, AND HOW IT WAS CLOSED

S6 made `countStoresByType` count drafts and suspensions, and the whole suite stayed green. The category tiles on the marketplace home would have advertised stores no visitor can reach.

This is the same failure mode the profile and channel milestones each hit once, and it is worth naming again: **the tests all approached that function through the marketplace pages, where only published stores exist anyway.** A count that included drafts produced the same number in every test that had ever been written. The fix is not a stricter assertion on an existing test but a test that creates the states nothing else creates — one live store, one draft, one suspended, of the same type — and demands the count be 1.

S6 is killed, and the whole matrix was then re-run end to end at 26 mutations: **no survivors**, worktree verified pristine afterwards.

NO MONEY MOVED

Explicitly **not implemented and not started**, as the milestone required: settlement, payouts, seller withdrawals, wallet movements, commission payment, referral payout, refund execution, payment capture logic, and any ledger posting.

The only money this milestone *reads* is `orders.status = 'paid'`, counted for ranking. It writes nothing to `orders`, `payments`, `ledger_entries`, `account_balances` or `refunds`. The payment state machine approved in phase 3 is untouched, and the OTP and authentication paths were not modified. Ledger reconciliation over a full capture → settlement → payout cycle is clean: **0 drift, 0 unbalanced transactions, net 0 XOF**.

Results

GATE	RESULT	PREVIOUS MILESTONE
<code>pnpm lint</code>	CLEAN	clean
<code>pnpm typecheck</code>	CLEAN	clean
<code>pnpm test</code>	583 passed · 16 skipped	581 passed · 16 skipped
Playwright	39 passed	33 passed
<code>pnpm build</code>	GREEN	green
Ledger reconciliation	CLEAN	clean
CI, GitHub Actions	3 / 3 JOBS GREEN — see the gate report	green
Store-engine mutations	26 / 26 killed	23 / 23
Horizontal overflow @ 390/768/1280/1680	NONE	—

All three CI jobs — dependency scan, lint · typecheck · test, and the browser assertions — pass on this branch's head. The exact run is named in the gate report accompanying this note.

The 16 skipped tests are the iPayMoney sandbox integration suite, gated on credentials that are not present in this environment. They are unchanged.

Three defects this milestone found in work that was already green

1

Twenty test fixtures inserted published stores with no publication date

The new `stores_published_has_timestamp` CHECK rejected them, in three e2e specs and the reconciliation probe. They had always been wrong — a published store with a null `published_at` sorts last forever — but nothing had ever asked.

2 A silent role grant in a test helper

`insert ... select ... from roles where key = $1` grants nothing when the key does not exist, and says so silently. A misspelled role produced an actor with no permissions, so every test asserting a *refusal* would have passed for entirely the wrong reason. The helper now checks the row count and throws.

3 A seller with no permission was invited into a wizard that would refuse them

The redesigned `/fr/sell` lost the "not yet a seller" state and offered everybody the create-store button. Both the screen and the route now check `store.create` — and the browser test types the wizard's URL directly to prove the route check is the one that matters.

What is deliberately not built

EXTENSION	WHERE IT LANDS	STATUS
Course structure	<code>products.kind = 'course'</code> + <code>course_modules</code>	LATER MILESTONE
Physical goods	<code>kind = 'physical'</code> + stock, weight, delivery zone	LATER MILESTONE
Services	<code>kind = 'service'</code> + availability, call-out radius	LATER MILESTONE
Delivery jobs	a job table keyed on the store	LATER MILESTONE
Settlement, payouts, refunds	the ledger, already built	NOT STARTED, BY INSTRUCTION

`products` is digital-only today (`kind = 'digital'` , enforced by a CHECK) and the engine does not pretend otherwise: a physical-product store can be created, published and found, and *selling* a physical item needs the product side extended. Each extension above is a new `kind` plus a table referencing `products` , behind the same authorization, the same publication rules and the same discovery queries. Nothing in this milestone has to be revisited to add one.

The three review decisions, and what each cost

#	DECISION	CHANGE
1	An empty store MAY be published. A store is a commercial identity, not a product container.	IMPLEMENTED see section I
2	Slug collisions stay refused. No auto-appended <code>-2</code> ; the seller is told and picks another name.	NO CODE CHANGE documented as deliberate
3	No admin console this milestone. The <code>seller</code> role stays an operator INSERT.	NO CODE CHANGE recorded as a pre-launch requirement

Remaining known issues

1 Every seller is enabled by a human with production database access.

Decision 3 keeps the SQL grant, which is workable while sellers are onboarded by hand and safe — it writes to the same `role_assignments` table `authorize()` reads, so nothing is bypassed. What it lacks is an audit trail naming *who* granted it and *why*. Documented in `store-engine.md` as a **pre-launch operational requirement**, not a permanent design.

2 `products` is digital-only.

A physical-product store can be created, published and found today — *selling* a physical item needs `kind` extended. The engine does not pretend otherwise; the empty storefront in section I is a physical-product store, and it says it has nothing for sale rather than implying a catalogue.

3 iPayMoney remains unimplemented and unreachable from this environment.

Unchanged by this milestone and untouched by it. The 16 gated sandbox tests still skip.

Recommended next milestone

RECOMMENDATION — THE DIGITAL PRODUCT ENGINE

Of the engines still to build, this is the one whose full path already exists end to end: a digital product can be created, priced in XOF, published, bought through the mock provider, paid for, and downloaded by exactly the buyer who paid — that journey is already under browser test. What it lacks is depth: versioned files, licence terms, download limits, a real library screen.

It is therefore the cheapest way to turn one *working* path into one *finished* one, and it needs nothing from iPayMoney — the mock provider covers it. The Formation Engine is the larger prize but wants course structure, progress and completion, which is a bigger surface to review at once.

Settlement should wait for the sandbox, since paying a seller before we can execute a refund is the wrong order to build in.

Review decision 1: an empty store may be published

The rule was: a seller could not publish until they had at least one offering. The review changed it. A store is a **commercial identity**, not a container that only becomes real once it holds stock — a tailor can print her address on a card and share it while she is still photographing her work.

WHERE THE RULE ACTUALLY LIVED

Nowhere in the domain. `publishStore` has no offering check and never had one; the requirement existed only because the dashboard rendered its publish control on `nextStep.key === "dash.stepPublishStore"`, and the guidance put "add an offering" first. The control is now tied to the store's own `status`. Tying a control to a *message* is how a product rule becomes unwritten, and it is why this one was never in a test.

What the permission costs, and what pays for it

A permissive rule has exactly two ways to be implemented wrongly: re-imposing the requirement somewhere else, and filling the emptiness with something invented. Both are now mutations, and both die.

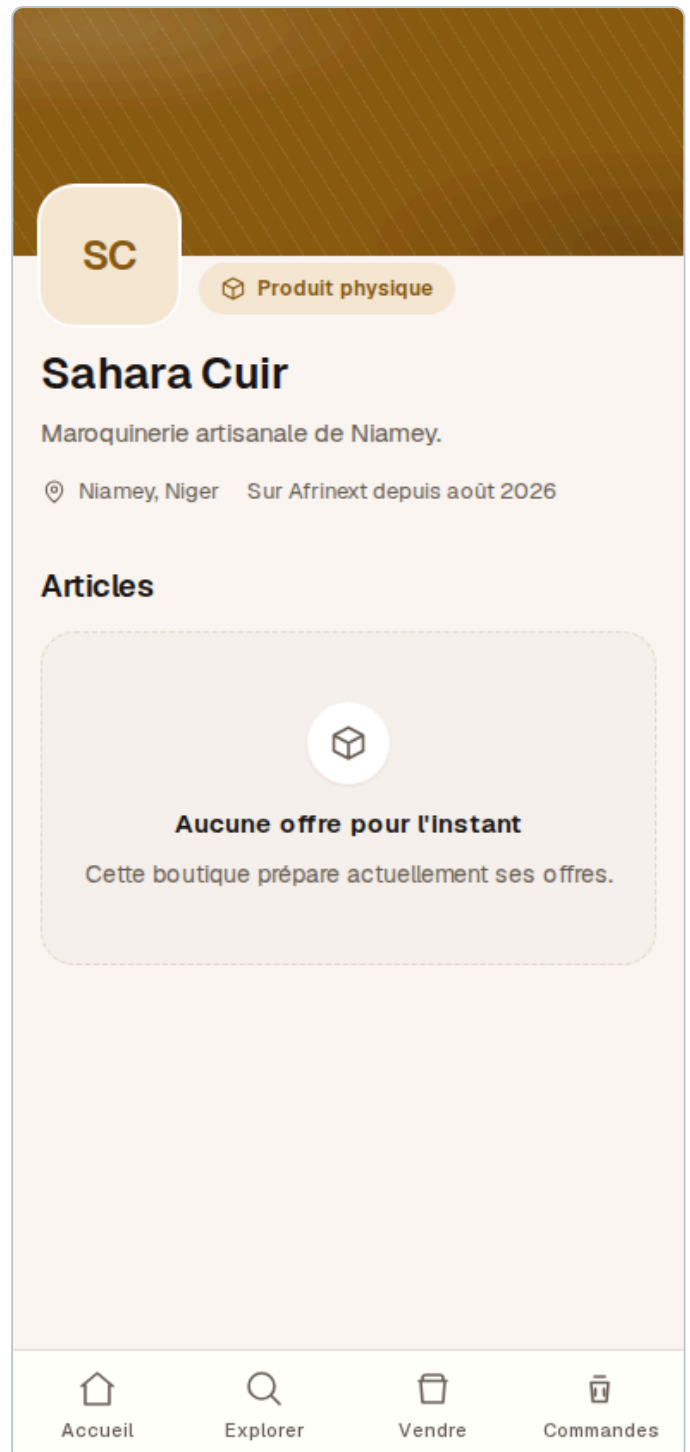
SURFACE	WITH ZERO PUBLISHED OFFERINGS
Public store page	"Aucune offre pour l'instant" · "Cette boutique prépare actuellement ses offres."
Marketplace card	"Aucune offre pour l'instant" — never a fabricated count
<code>listPublicProducts</code>	<code>[]</code> — never a placeholder row
<code>discoverOfferings</code>	contributes nothing
<code>countStoresByType</code>	counts it once; it is a real store



An empty draft. "Publiez votre boutique" is now the first step, and the control is there — on a store with nothing in it at all. Vocabulary still follows the trade: this is a physical-product store, so its offerings are *Articles*.

The adversarial tests

The domain test asserts both halves in one place, because the interesting failure is not "it refused to publish" but "it published and then invented something":



Published, and empty. A real identity — cover, monogram, tagline, place, member-since — and then the truth. No placeholder product, no price, no "coming soon" item in the list.

ASSERTION	WHAT IT WOULD CATCH
publishes with zero products; <code>status</code> and <code>published_at</code> are real	the requirement re-imposed
<code>findPublicStore</code> finds it; <code>discoverStores</code> lists it	published but invisible
<code>listPublicProducts</code> returns <code>[]</code>	a placeholder row
<code>offeringCount</code> is 0	a decorative count
<code>discoverOfferings</code> yields nothing for it	a phantom in the offerings feed
<code>select count(*) from products</code> is still 0 <i>after</i> publishing	publishing writing a product row
a stranger cannot publish it; a suspended empty store cannot publish	the permission having loosened something else

The browser suite then does it anonymously against a production build: publishes an empty store through the real wizard and the real dashboard, fetches its public URL with no cookies, asserts the empty state is on the page, asserts **no XOF appears anywhere**, and asserts no link points at an offering under that store.

AND THE ASSERTION WAS CHECKED FOR VACUITY

A test that passes proves nothing until it can fail. The French empty-state copy was temporarily replaced with a placeholder, the app rebuilt, and the catalog journey re-run: it failed on exactly the intended line — `getText('prépare actuellement ses offres')`, element not found. Copy restored, rebuilt, green again.